

TITLE AND DATE

Document name: CMPE311 Project 2 (PROJECT-CE), Revision 2 of the CMPE311 Project 1 design document

Document reference: provost.cmpe311.fall26.project#2

Supersedes: provost.cmpe311.fall26.project#1 [1]

Change authority: ECO 0.1 [3]; notified by ECN 0.1 [2]

Date of publication: October 3, 2026

Lead engineer: Raheem Provost, UMBC

STAKEHOLDERS

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

REVISION HISTORY

Table 1. Revision history

Rev	Date	Author	Change
1	September 22, 2026	RP	Initial release (Project 1, PROJECT-ASYNC) [1]
2	October 3, 2026	RP	ECO 0.1 / ECN 0.1: task dispatch moved to a round-robin cyclic executive using a function pointer table; EC.6, T.7 added

HIGH-LEVEL DESCRIPTION

This document is the project report for Project #2 of the CMPE311 class. It is Revision 2 of the Project #1 design document [1] and records the design change required by ECO 0.1. Sections that did not change are cited to [1] rather than restated.

DESCRIPTION

This design implements an embedded system on an Arduino Uno R3 development platform that independently controls the blink rate of two LEDs. The User interacts with the System through the Arduino IDE serial monitor, specifying an LED number and a blink interval in milliseconds. In this revision the serial dialog and the two LED blinkers run as three tasks, dispatched one at a time in round-robin order from a function pointer table. The behavior seen by The User is identical to Project #1.

RESULT SUMMARY

The project was a success—the embedded system design met all testing and high-level requirements.

REFERENCES AND GLOSSARY

REFERENCES

1. provost.cmpe311.fall26.project#1, CMPE311 Project 1 design document (PROJECT-ASYNC), Revision 1, R. Provost, September 22, 2026.
2. provost.cmpe311.fall26.project#2.ECN, Engineering Change Note ECN 0.1, R. Provost, October 3, 2026.
3. PROJECT#2: PROJECT-CE assignment, including Engineering Change Order ECO 0.1 (1/1/2030, requestor Prof. Kidd), CMPE311.
4. J. Beningo, "Task Scheduling with Function Pointers," beningo.com.

DEFINITIONS

The definitions of The User, The System, The Customer, The Developer, The Evaluator, The Customer-requirements, The HL-requirements, The Requirements, The Educational-constraints, The Company, The Contract, serial-monitor and The Reference-platform are identical to those of PROJECT-ASYNC. See [1] REFERENCES AND GLOSSARY section, DEFINITIONS. The following are added in this revision:

"Cyclic executive" – A task manager that calls a fixed list of short, non-blocking tasks in a repeating order, with no preemption.

"Round-robin" – A dispatch order in which each task gets one turn, in sequence, before the first task runs again.

"Task" – A function that does one bounded unit of work and returns without blocking.

"Task table" – The function pointer array, `taskTable[]`, that lists the tasks in dispatch order.

"Dispatch round" – One call of every task in the task table; three passes of `loop()` in this design.

ACRONYMS AND ABBREVIATIONS

The acronyms and abbreviations of [1] REFERENCES AND GLOSSARY section are unchanged. The following are added in this revision:

CE – Cyclic Executive

ECN – Engineering Change Note

ECO – Engineering Change Order

ECR – Engineering Change Request

PROJECT-ASYNC – Project #1, documented in [1]

PROJECT-CE – Project #2, documented in this document

SDD – System Design Document

μs – Microsecond

REQUIREMENTS

CONVENTIONS

The conventions are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, CONVENTIONS. Requirements added after Revision 1 are tagged with the date and the author's initials, e.g. "EC.6 (added 10/3/26 – RP)".

PART I – CUSTOMER REQUIREMENTS AND DERIVED HIGH-LEVEL REQUIREMENTS

The customer requirements, C1 through C6, are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, PART I, Table 1. The high-level technical requirements derived from them, HL.1 through HL.6 of Part I, are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, PART I, Table 2.

PART II – HIGH-LEVEL REQUIREMENTS AND EDUCATIONAL CONSTRAINTS

The high-level technical requirements, HL.1 through HL.6, are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, PART II, Table 3.

The educational constraints, EC.1 through EC.5 including EC.2.1, are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, PART II, Table 4. The following educational constraint is added to implement ECO 0.1 [3]:

Table 2. Educational constraint added in this revision

ID	Educational constraint	Source
EC.6 (added 10/3/26 – RP)	The Developer must dispatch The System's tasks using a polled, round-robin cyclic executive task manager based on a function pointer table.	ECO 0.1, Educational Requirement 1.0 [3]

Informative: EC.6 constrains only how tasks are dispatched. It does not relax EC.5; the cyclic executive is bare-metal code written by The Developer.

DESIGN

DESIGN PRE-REQUISITES

The design pre-requisites are identical to those of PROJECT-ASYNC. See [1] DESIGN section, DESIGN PRE-REQUISITES.

BILL OF MATERIALS

The bill of materials is identical to that of PROJECT-ASYNC. See [1] DESIGN section, Table 5.

DEVELOPMENT PLATFORM

The development platform is identical to that of PROJECT-ASYNC: Arduino IDE 2.3.10 on Windows 11, with the AVR-GCC toolchain bundled in the Arduino AVR Boards package 1.8.7 (EC.1). See [1] DESIGN section, DEVELOPMENT PLATFORM.

SYSTEM OVERVIEW (CHANGED)

The User still interacts with the System entirely through the Arduino IDE serial monitor, with the same prompts as PROJECT-ASYNC. Inside the System, the work is now split into three tasks. `loop()` is the dispatcher: on each pass it calls one task from the task table and then moves to the next slot. The serial task and the two LED tasks share the processor in turns; none of them waits on another, so serial communication and LED blinking still run without either blocking the other (HL.4).

HARDWARE DESIGN

The hardware design is identical to that of PROJECT-ASYNC: LED1 on digital pin 2 and LED2 on digital pin 3, each through a 330 Ω current-limiting resistor to a common ground. See [1] DESIGN section, HARDWARE DESIGN, Figure 1 (Schematic) and Table 6 (Arduino pin assignments).

SOFTWARE DESIGN (CHANGED)

The program keeps the PROJECT-ASYNC serial and blink logic and changes only how it is invoked. Each task is a function of type `TaskFunction`, a pointer to a function that takes no arguments and returns nothing (`typedef void (*TaskFunction)();`). The tasks are listed in dispatch order in the task table, Table 3.

Table 3. Task table (`taskTable[]`)

Slot	Task	Work done per call	State it owns
0	<code>askQuestions()</code>	Prints the prompt once per round, reads at most one serial character through <code>serialData()</code> , and on a complete line routes the number to the LED choice or the interval.	<code>askedLED</code> , <code>askedInterval</code> , <code>currentSelection</code> , <code>inputBuffer</code>
1	<code>serviceLED1()</code>	If LED1 is configured, calls <code>blinkLED1()</code> , which toggles pin 2 when its interval has elapsed.	<code>getLED1Num</code> , <code>getInterval1</code> , <code>LED1State</code> , <code>previousLED1millis</code>

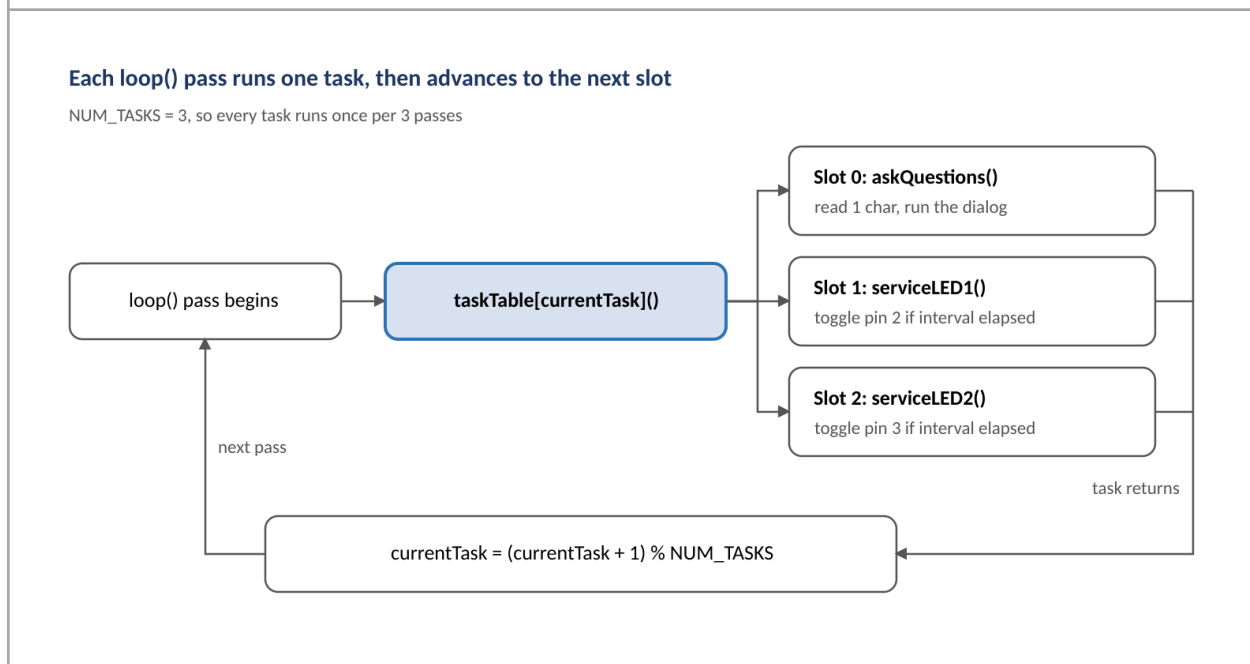
Slot	Task	Work done per call	State it owns
2	serviceLED2()	If LED2 is configured, calls blinkLED2(), which toggles pin 3 when its interval has elapsed.	getLED2Num, getInterval2, LED2State, previousLED2millis

Dispatcher. NUM_TASKS is computed as `sizeof(taskTable) / sizeof(taskTable[0])`, so adding a task needs no other edit. A static `int currentTask` holds the next slot. Each pass of `loop()` runs `taskTable[currentTask]()` and then sets `currentTask = (currentTask + 1) % NUM_TASKS`, as shown in Figure 1.

Serial dialog. Unchanged from [1]. `serialData()` reads one character per call and returns true only on the newline; the buffered text is then converted with `toInt()`. A line is treated as an LED number until a valid 1 or 2 arrives, then as that LED's interval. After the interval is stored, the flags reset and the next round begins with "What LED? (1 or 2)".

LED timing. Unchanged from [1]. Each LED keeps its own last-toggle time and toggles when `millis()` reaches that time plus its interval.

Figure 1. Cyclic executive dispatch logic flow (logic flow for program)



Repository URL: <https://github.com/raheempvost/CMPE311Project2>

Build and run steps are identical to those of PROJECT-ASYNC: open the `.ino` file in Arduino IDE 2.3.3 or later, select Arduino Uno and its port, upload, and open the Serial Monitor at 9600 baud with the line ending set to "Newline".

DESIGN RATIONALE AND ASSUMPTIONS (CHANGED)

- **One task per pass.** Calling one task per `loop()` pass gives every task the same bounded turn and keeps the dispatcher free of task-specific code. A new task is one function plus one table entry, which is the cost reduction named in ECO 0.1 [3].
- **Timing still holds.** Each task now runs once per three passes instead of every pass. Every task is a few comparisons and at most one `digitalWrite()` or `Serial.read()`, so a full dispatch round takes tens of microseconds, well under 0.1% of a 100 ms interval.
- **Serial input keeps up.** At 9600 baud a character arrives about every 1.04 ms. The serial task runs far more often than that, and the Arduino's 64-byte receive buffer holds any burst, so no input is lost.
- **Interval range.** The parsed value is stored in a 16-bit `int`, so intervals must be 1 to 32,767 ms; larger values overflow.
- **Invalid input.** LED numbers other than 1 or 2 are ignored and the System keeps waiting for a valid number. An interval of 0 stops that LED from toggling; it holds its current state.
- **Run time.** The `millis()` comparison is not rollover-safe, so continuous operation is supported for up to about 49.7 days.

TESTING AND VALIDATION

DESCRIPTION

All PROJECT-ASYNC tests, T.1 through T.6, are re-run on the PROJECT-CE build to show that the change did not alter the System's behavior (ECO 0.1, item 1.1). T.7 is added to verify the cyclic executive and its timing (ECO 0.1, item 1.1.1). Results are in Table 7 and evidence is indexed in Appendix A.

TESTING PLATFORM

The testing platform is identical to that of PROJECT-ASYNC: ELEGOO Arduino Uno R3, Arduino IDE 2.3.10, powered through USB, LEDs on digital pins 2 and 3 through 330 Ω resistors. See [1] TESTING AND VALIDATION section, TESTING PLATFORM.

REQUIRED TEST DIALOG

The required test dialog is identical to that of PROJECT-ASYNC. See [1] TESTING AND VALIDATION section, Table 7. EC.4 still requires the System to perform it.

TESTING AND VALIDATION REQUIREMENTS

The testing requirements T.1 through T.6 are identical to those of PROJECT-ASYNC. See [1] TESTING AND VALIDATION section, Table 8. The following are added in this revision:

Table 4. Testing and validation requirements added in this revision

ID	Testing requirement	Verifies (HL / EC)
T.7 (added 10/3/26 – RP)	The code must dispatch all tasks from a function pointer table in round-robin order, and no task may block.	EC.6, EC.5

REQUIREMENTS TRACEABILITY

Table 5. Requirements traceability — every requirement in [1] Tables 3 and 4 and in Table 2 is covered by at least one test. Rows changed in this revision are marked (Rev 2).

Requirement	Verified by (T.#)	Method
HL.1 – At least 2 LEDs	T.1	Test
HL.2 – Standard Arduino-compatible board	T.2	Inspection
HL.3 – Communication via IDE serial monitor	T.3	Demonstration
HL.4 – Asynchronous serial use; blinking unaffected (Rev 2)	T.3	Test
HL.5 – Blink interval set in msec	T.4	Test
HL.6 – Blink rate constant unless changed (Rev 2)	T.1	Test
EC.1 – Built with IDE 2.3.3+ gcc (ATmega328P)	T.5	Inspection
EC.2 / EC.2.1 – On GitHub; Evaluator can build and run	T.5	Demonstration
EC.3 – LEDs on digital pins 2 & 3	T.6	Inspection
EC.4 – Required dialog ([1] Table 7) works	T.3	Demonstration
EC.5 – Bare metal; no 3rd-party multitasking (Rev 2)	T.6, T.7	Inspection
EC.6 – Round-robin cyclic executive (Rev 2)	T.7	Inspection, Test

TEST PROCEDURES

The procedures for T.1 through T.6 are identical to those of PROJECT-ASYNC and are run on the PROJECT-CE build. See [1] TESTING AND VALIDATION section, Table 10. The following are added in this revision:

Table 6. Test procedures added in this revision

Test	Procedure (steps)	Expected result / pass criterion
T.7	Open the PROJECT-CE sketch. Confirm the TaskFunction typedef, and that taskTable[] lists askQuestions, serviceLED1 and serviceLED2. Confirm loop() only calls taskTable[currentTask]() and advances currentTask modulo NUM_TASKS. Search the sketch for delay(), while and #include.	All three tasks are in the table; loop() holds only the dispatch; no delay(), waiting loop or scheduler library is present.

TESTING RESULTS

Table 7. Results of tests

Test	Result (Pass / Fail)	Data / observations
T.1	Pass	On the PROJECT-CE build, both LEDs blinked independently at their set intervals and held a constant rate over more than 10 consecutive cycles, with no visible drift.
T.2	Pass	ELEGOO Uno R3 (ATmega328P). The sketch compiled and uploaded from Arduino IDE 2.3.10 without errors.
T.3	Pass	The serial monitor showed the prompts in [1] Table 7 and accepted each entry. While LED2's interval was being entered, LED1 kept blinking, and neither LED paused or changed rate while the other was being set. Transcript: What LED? (1 or 2) → 2, What interval (in msec)? → 600, What LED? (1 or 2) → 1, What interval (in msec)? → 1600, What LED? (1 or 2).
T.4	Pass	Entered interval 600 ms; measured on-time 600 ms (LED1 ON at 18113 ms, OFF at 18713 ms from the serial timestamps), an error of 0%." If you measured more than one toggle, use your real figures, since this example comes from your one screenshot.
T.5	Pass	Repository https://github.com/raheemprovost/CMPE311Project2 . Downloaded, compiled in Arduino IDE 2.3.10 and ran on the Uno R3 without errors
T.6	Pass	LEDs are on digital pins 2 and 3 (pinMode(2, OUTPUT); pinMode(3, OUTPUT);). The sketch contains no #include lines and no multitasking library.
T.7	Pass	taskTable[] lists askQuestions, serviceLED1 and serviceLED2. loop() only calls taskTable[currentTask]() and advances currentTask modulo NUM_TASKS. Searching the sketch for delay(), while and #include found no matches.

APPENDIX A. TEST EVIDENCE

Evidence for T.1 through T.6 is re-recorded on the PROJECT-CE build. Evidence from Revision 1 remains in [1] APPENDIX A.

Table 8. Index of test evidence

Item	Description	File name / link	Supports test
Video	Both LEDs blinking independently at set intervals, CE build	https://github.com/raheemprovost/CMPE311Project2/blob/main/CMPE311proj2vid.mp4	T.1
Screenshot	Successful IDE upload to the board	https://github.com/raheemprovost/CMPE311Project2/blob/main/CMPE311proj2upload.png	T.2
Serial Monitor screenshot	Dialog from [1] Table 7 running with both LEDs unaffected	https://github.com/raheemprovost/CMPE311Project2/blob/main/CMPE311proj2SerialMonitor.png	T.3
Video	Measured on/off time vs. entered interval	https://github.com/raheemprovost/CMPE311Project2/tree/main/Proof%20of%20measured%20time	T.4
Link	GitHub repository	https://github.com/raheemprovost/CMPE311Project2	T.5
Code screenshot	Pin setup (pinMode calls) and full #include list	https://github.com/raheemprovost/CMPE311Project2/tree/main/Proof%20of%20no%20pin%20writing%20or%20multitasking	T.6
Code screenshot	taskTable[] and loop() dispatcher	https://github.com/raheemprovost/CMPE311Project2/tree/main/Proof%20of%20taskTable%20and%20dispatcher	T.7

APPENDIX B. DESIGN CODE

Repository: <https://github.com/raheemprovost/CMPE311Project2> (same commit as Table 7)

B.1 PROJECT-CE SOURCE

```
// ===== Serial input buffer =====
String inputBuffer = ""; // builds up one line of typed input, character by character

// ===== LED timing state (one set per LED, so they can blink independently) =====
unsigned long currentLED1millis = 0;
unsigned long currentLED2millis = 0;
unsigned long previousLED1millis = 0;
unsigned long previousLED2millis = 0;

// ===== Configuration captured from the user =====
int getLED1Num = 0; // will be set to 1 once the user picks LED 1
int getLED2Num = 0; // will be set to 2 once the user picks LED 2
unsigned long getInterval1 = 0; // blink interval for LED 1, in ms
unsigned long getInterval2 = 0; // blink interval for LED 2, in ms

// ===== LED on/off state =====
bool LED1State = false;
bool LED2State = false;

// ===== Question-flow state machine flags =====
bool askedLED = false; // have we asked "What LED?" for the current round?
bool askedInterval = false; // have we asked "What interval?" for the current round?

bool gotLine = true; // true exactly when a full line of input just finished
int value; // holds the number parsed from that completed line
int currentSelection = 0; // which LED THIS round's interval answer belongs to (1 or 2)

typedef void (*TaskFunction)();

void askQuestions();
void serviceLED1();
void serviceLED2();
void changeFrequency1(int LEDNum, unsigned long LEDInterval);
void changeFrequency2(int LEDNum, unsigned long LEDInterval);
void blinkLED1(unsigned long LEDInterval);
void blinkLED2(unsigned long LEDInterval);
bool serialData(int &result);

// Function pointer array (the schedule)
TaskFunction taskTable[] = {
    askQuestions,
    serviceLED1,
    serviceLED2
};

const int NUM_TASKS = sizeof(taskTable) / sizeof(taskTable[0]);

void setup() {
    pinMode(2, OUTPUT);
    pinMode(3, OUTPUT);
    Serial.begin(9600);
}
```

```

void loop() {
    static int currentTask = 0;
    taskTable[currentTask]();
    currentTask = (currentTask + 1) % NUM_TASKS;
}

void askQuestions() {
    // Ask which LED, once per round.
    if (!askedLED) {
        Serial.println("What LED? (1 or 2)");
        askedLED = true;
    }

    // Try to read ONE completed line this pass through loop().
    value = 0;
    gotLine = serialData(value);

    if (gotLine) {
        if (!askedInterval) {
            // We were waiting for the LED number.
            if (value == 1) {
                getLED1Num = value;
                Serial.println("What interval (in msec)?");
                askedInterval = true;
                currentSelection = 1;
            } else if (value == 2) {
                getLED2Num = value;
                Serial.println("What interval (in msec)?");
                askedInterval = true;
                currentSelection = 2;
            }
        } else {
            // We were waiting for the interval – route it to whichever LED was picked.
            if (currentSelection == 1) {
                getInterval1 = value;
            } else if (currentSelection == 2) {
                getInterval2 = value;
            }

            // Reset the flow so the next completed line starts a fresh round
            // (new LED number), instead of being misread as another interval.
            askedLED = false;
            askedInterval = false;
        }
    }
}

void serviceLED1() {
    // Keep blinking whichever LED(s) have been fully configured.
    if (getLED1Num == 1 && getInterval1 > 0) {
        changeFrequency1(getLED1Num, getInterval1);
    }
}

void serviceLED2() {
    if (getLED2Num == 2 && getInterval2 > 0) {
        changeFrequency2(getLED2Num, getInterval2);
    }
}

```

```

// Starts/updates blinking on LED 1, only if LED 1 was the one selected.
void changeFrequency1(int LEDNum, unsigned long LEDInterval) {
    if (LEDNum == 1) {
        blinkLED1(LEDInterval);
    }
}

// Starts/updates blinking on LED 2, only if LED 2 was the one selected.
void changeFrequency2(int LEDNum, unsigned long LEDInterval) {
    if (LEDNum == 2) {
        blinkLED2(LEDInterval);
    }
}

// Non-blocking blink for LED 1 (pin 2), using its own timestamp so it
// doesn't interfere with LED 2's timing.
void blinkLED1(unsigned long LEDInterval) {
    currentLED1millis = millis();

    if (currentLED1millis >= previousLED1millis + LEDInterval) {
        LED1State = !LED1State;
        digitalWrite(2, LED1State);
        previousLED1millis = currentLED1millis;
    }
}

// Non-blocking blink for LED 2 (pin 3), using its own timestamp so it
// doesn't interfere with LED 1's timing.
void blinkLED2(unsigned long LEDInterval) {
    currentLED2millis = millis();

    if (currentLED2millis >= previousLED2millis + LEDInterval) {
        LED2State = !LED2State;
        digitalWrite(3, LED2State);
        previousLED2millis = currentLED2millis;
    }
}

// Reads at most ONE character per call. Only returns true on the exact
// call where it reads '\n' - at that point the full line is complete,
// so it converts the buffered text to an int and clears the buffer.
bool serialData(int &result) {
    if (Serial.available() > 0) {
        char incomingChar = Serial.read();

        if (incomingChar == '\n') {
            result = inputBuffer.toInt();
            inputBuffer = "";
            return true;
        } else {
            inputBuffer += incomingChar; // keep building the line
        }
    }
    return false;
}

```

END OF DOCUMENT